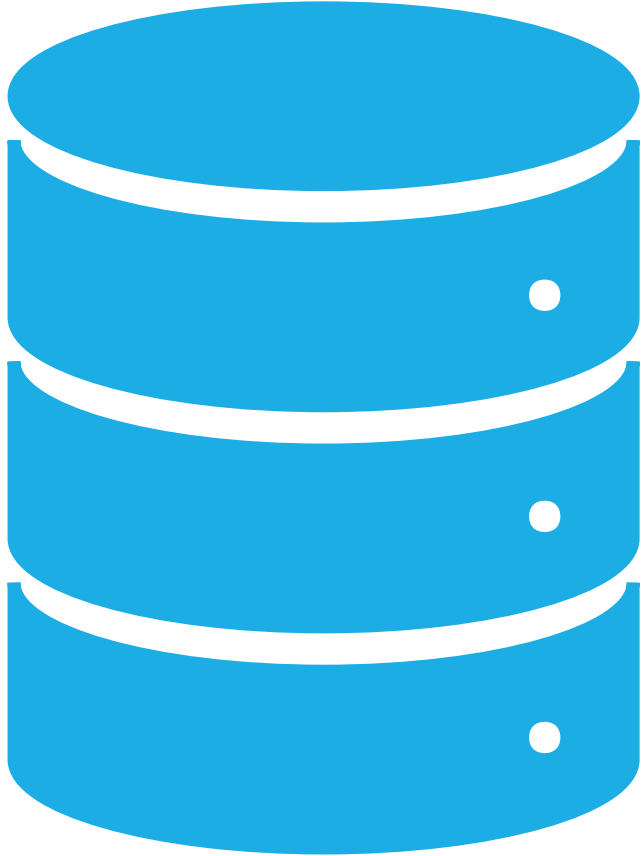




Python + SQL Project

BY: DEEPTI SINGH



OBJECTIVE

This project analyses an e-commerce dataset, using SQL for data extraction and querying and Python for advanced analysis and visualization. The objective is to generate actionable business insights across sales performance, customer behaviour, and customer retention, supporting data-driven decision-making.



For the **Python-based data analysis**, I cleaned the dataset by merging all tables into a single consolidated table. I also removed duplicate records and eliminated unnecessary data to ensure accuracy and consistency before performing the analysis.



For the **SQL-based data analysis**, I uploaded the tables in their original structure without modification and conducted the analysis directly using SQL queries.

Methodology for Python and SQL Data Analysis



BASIC PROBLEMS

A. List all unique cities where customers are located: Python Answer:

```
▶ Unique_customer_city = sorted(df_customers['customer_city'].unique())  
print("Number of Unique Cities where Customers are located:", len(Unique_customer_city))  
  
for city in Unique_customer_city:  
    print(city)
```

```
*** Number of Unique Cities where Customers are located: 4119  
abadia dos dourados  
abadiania  
abaete  
abaetetuba  
abaiara  
abaira  
abare  
abatia  
abdon batista  
abelardo luz  
abrantres  
abre campo  
abreu e lima  
acaiaca  
acailandia  
acajutiba  
acarau  
acari  
acegua  
aconiara
```

SQL Answer

```
10
11      /*1. List all unique cities where customers are located. */
12 •    select distinct(customer_city) from customers
13      order by customer_city;
14
```

<	
Result Grid	
Filter Rows:	
Export: 	
Wrap Cell Content: 	
Fetch rows: 	
	customer_city
▶	Abadia Dos Dourados
	Abadiania
	Abaete
	Abaetetuba
	Abaiara
	Abaira
	Abare
	Abatia
	Abdon Batista
	Abelardo Luz
	Abrantes
	Abre Campo
	Abreu E Lima

B. Count the number of orders placed in 2017

- ✓ B. Count the number of orders placed in 2017

```
orders_2017= (df_merged[df_merged['order_purchase_timestamp'].dt.year== 2017]).copy()  
orders_2017  
print("Count of Orders placed in 2017: ", len(orders_2017))  
unique_orders_count = orders_2017['order_id'].nunique()  
print("Count of Unique Orders placed in 2017: ", unique_orders_count)
```

- ✓
Count of Orders placed in 2017: 53539
Count of Unique Orders placed in 2017: 44579

Chart for the Number of orders placed in 2017



SQL Answer

```
18 FROM orders;
19
20 • select year(order_purchase_timestamp), count(order_id) from orders
21 where year(order_purchase_timestamp) = 2017
22 group by year(order_purchase_timestamp);
```



Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	year(order_purchase_timestamp)	count(order_id)
▶	2017	45101

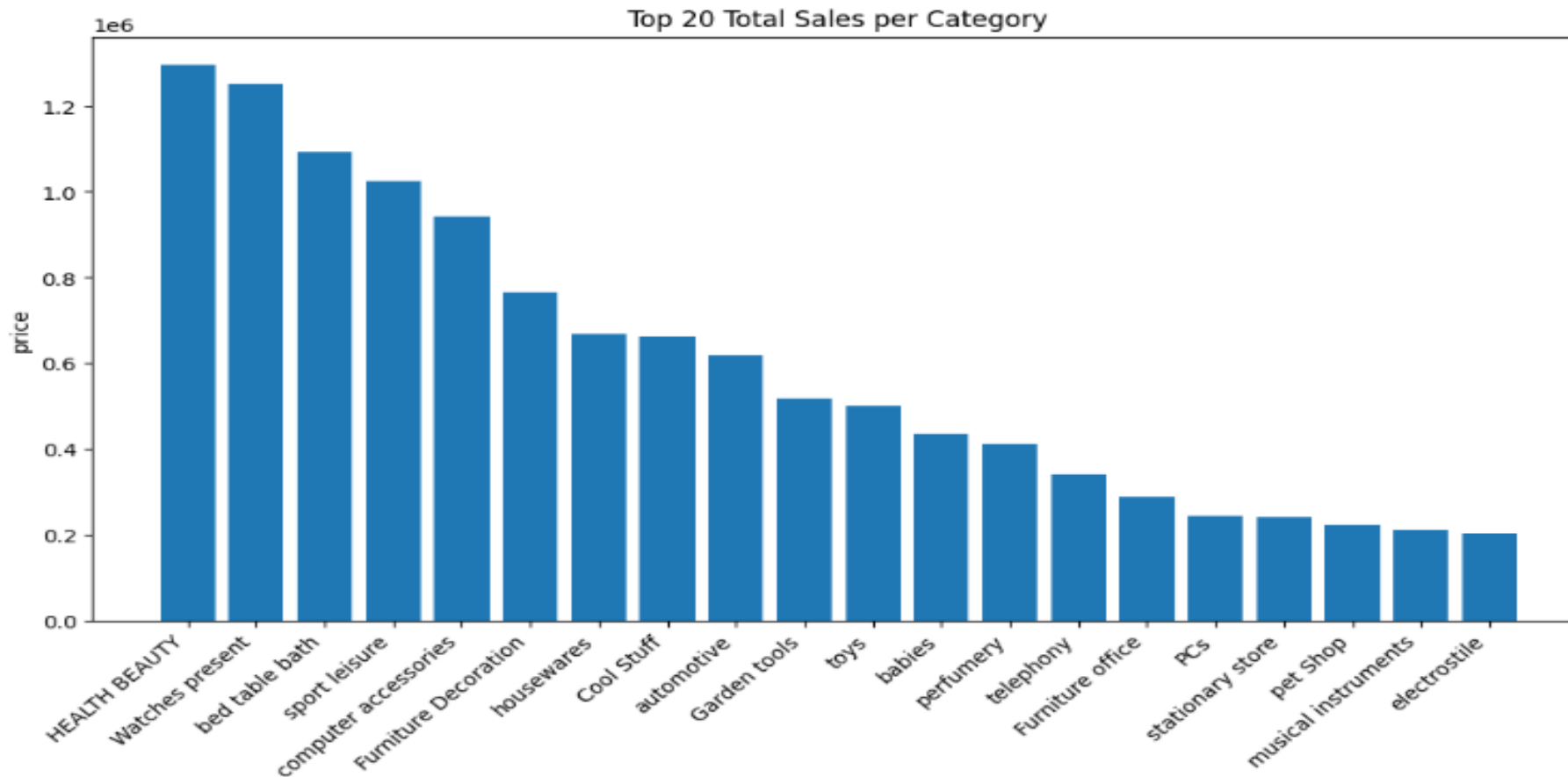
C. Find the total sales per category

C. Find the total sales per category

```
sales_per_category= df_merged.groupby('product_category')['price'].sum().reset_index()  
sales_per_category= sales_per_category.sort_values(by='price', ascending=False)  
sales_per_category
```

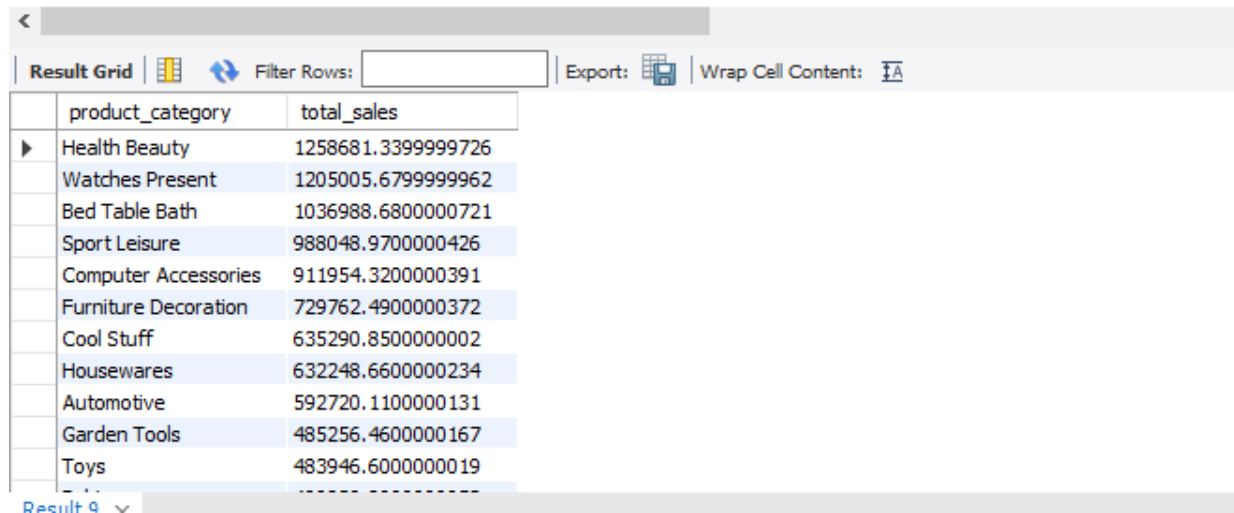
	product_category	price
30	HEALTH BEAUTY	1297490.77
45	Watches present	1253143.30
49	bed table bath	1092551.02
68	sport leisure	1023996.34
53	computer accessories	942277.57
...	...	...
58	flowers	1110.04
32	House Comfort 2	773.17
50	cds music dvds	730.00
18	Fashion Children's Clothing	569.85

Chart for the total sales per category



SQL Answer

```
28  /*Find the total sales per category*/
29
30  •  select p.product_category, sum(o.price) as total_sales from order_items as o
31      join products as p
32      on o.product_id = p.product_id
33      group by p.product_category
34      order by total_sales desc;
```



	product_category	total_sales
▶	Health Beauty	1258681.33999999726
	Watches Present	1205005.67999999962
	Bed Table Bath	1036988.68000000721
	Sport Leisure	988048.97000000426
	Computer Accessories	911954.32000000391
	Furniture Decoration	729762.49000000372
	Cool Stuff	635290.85000000002
	Housewares	632248.66000000234
	Automotive	592720.11000000131
	Garden Tools	485256.46000000167
	Toys	483946.6000000019

Result 0

D. Calculate the percentage of orders that were paid in instalments

D. Calculate the percentage of orders that were paid in installments

```
#Total Number of Unique Orders
total_no_of_orders= df_merged['order_id'].nunique()
total_no_of_orders

#Total Number of Unique Orders paid in Installments
orders_installments= df_merged[df_merged['payment_installments'].notna() & (df_merged['payment_installments'] > 1)]['order_id'].nunique()
orders_installments

#percentage of Orders that were paid in installments
percentage_installments= (orders_installments/total_no_of_orders)*100
percentage_installments= round(percentage_installments, 2)
percentage_installments

print("Total Number of Unique Orders Placed: ", total_no_of_orders)
print("Total Number of Unique Orders Paid in Installments: ", orders_installments)
print("Percentage of Orders Paid in Installments: ", percentage_installments,"%")
```

```
Total Number of Unique Orders Placed: 99441
Total Number of Unique Orders Paid in Installments: 51170
Percentage of Orders Paid in Installments: 51.46 %
```

SQL Answer

```
35
36      /*Calculate the percentage of orders that were paid in installments*/
37
38 • ○ select round( count(distinct case when payment_installments >1 then order_id end)*100
39      /count(distinct order_id) , 2) as Percentage_installments_orders from payments;
40
```



Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	Percentage_installments_orders
▶	51.46

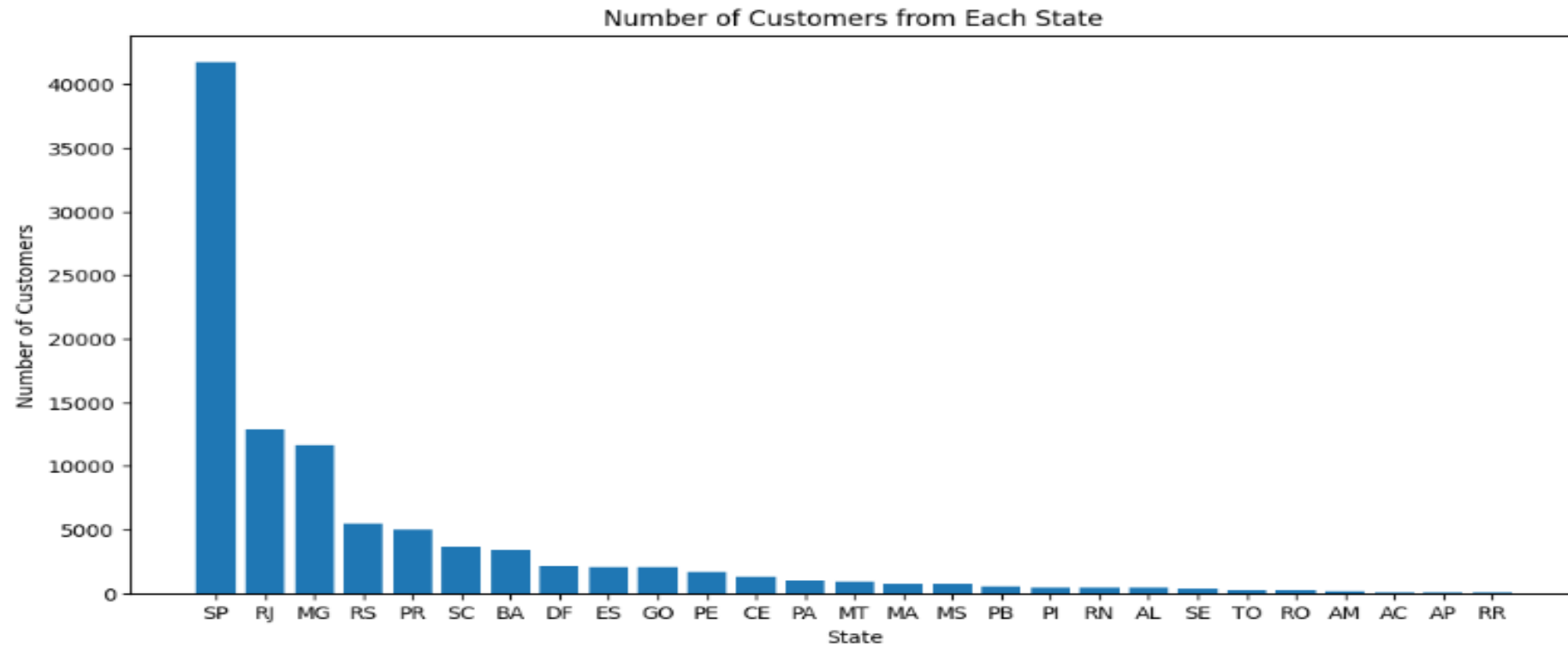
E. Count the number of customers from each state

- E. Count the number of customers from each state

```
1 customer_state_counts= df_customers['customer_state'].value_counts().reset_index()  
customer_state_counts
```

	customer_state	count
0	SP	41746
1	RJ	12852
2	MG	11635
3	RS	5466
4	PR	5045
5	SC	3637
6	BA	3380
7	DF	2140
8	ES	2033
9	GO	2020
10	PE	1652

Chart for number of customers from each state



SQL Answer

```
41      /*Count the number of customers from each state*/
42  •    select customer_state, count(customer_id) as total_customers
43      from customers
44      group by customer_state
45      order by count(customer_id) desc;
46
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 		
	customer_state	total_customers
▶	SP	41746
	RJ	12852
	MG	11635
	RS	5466
	PR	5045
	SC	3637
	BA	3380
	DF	2140
	ES	2033
	GO	2020
	PE	1652
	--	----



INTERMEDIATE PROBLEMS

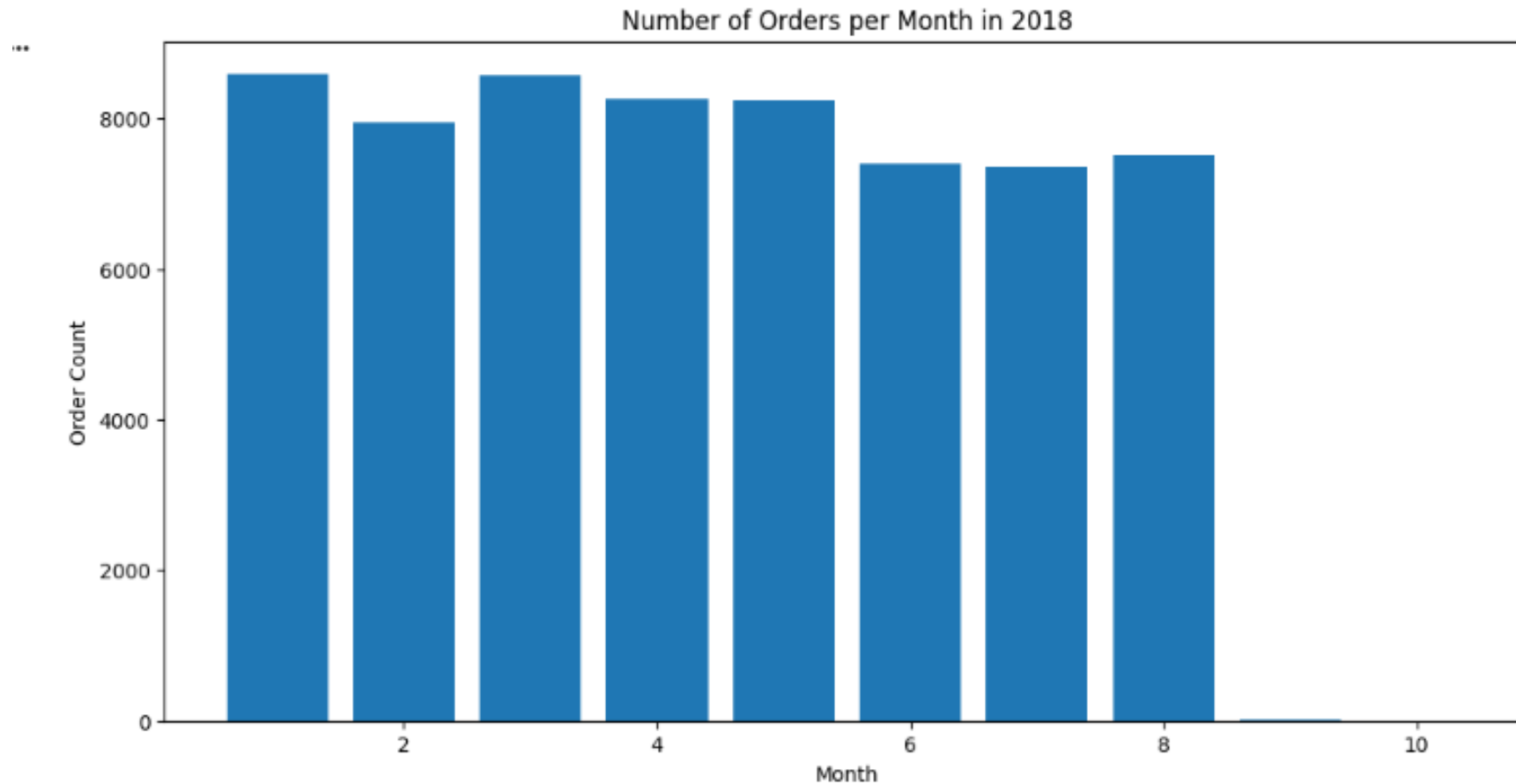
A. Calculate the number of orders per month in 2018

✓ A. Calculate the number of orders per month in 2018

```
orders_2018= df_merged[df_merged['order_purchase_timestamp'].dt.year== 2018]
order_count_2018= orders_2018.groupby(orders_2018['order_purchase_timestamp'].dt.month).size().reset_index(name='order_count')
order_count_2018
```

	order_purchase_timestamp	order_count
0	1	8597
1	2	7956
2	3	8567
3	4	8266
4	5	8248
5	6	7397
6	7	7351
7	8	7522
8	9	16
9	10	4

Chart for number of orders per month in 2018



SQL Answer

```
47      /*Calculate the number of orders per month in 2018*/
48 •    select extract(year from order_purchase_timestamp) as Year_extracted,
49           extract(month from order_purchase_timestamp) as month_extracted,
50           count(customer_id) as Number_of_customers
51      from orders
52     where extract(year from order_purchase_timestamp) = 2018
53     group by Year_extracted, month_extracted
54     order by month_extracted;
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	Year_extracted	month_extracted	Number_of_customers
▶	2018	1	7269
	2018	2	6728
	2018	3	7211
	2018	4	6939
	2018	5	6873
	2018	6	6167
	2018	7	6292
	2018	8	6512
	2018	9	16
	2018	10	4

B. Find the average number of products per order, grouped by customer city.

B. Find the average number of products per order, grouped by customer city.

```
avg_products_per_city= df_merged.groupby(['customer_city', 'order_id'])['product_id'].count().reset_index(name='avg_products_per_order_per_city' )
avg_products_per_city

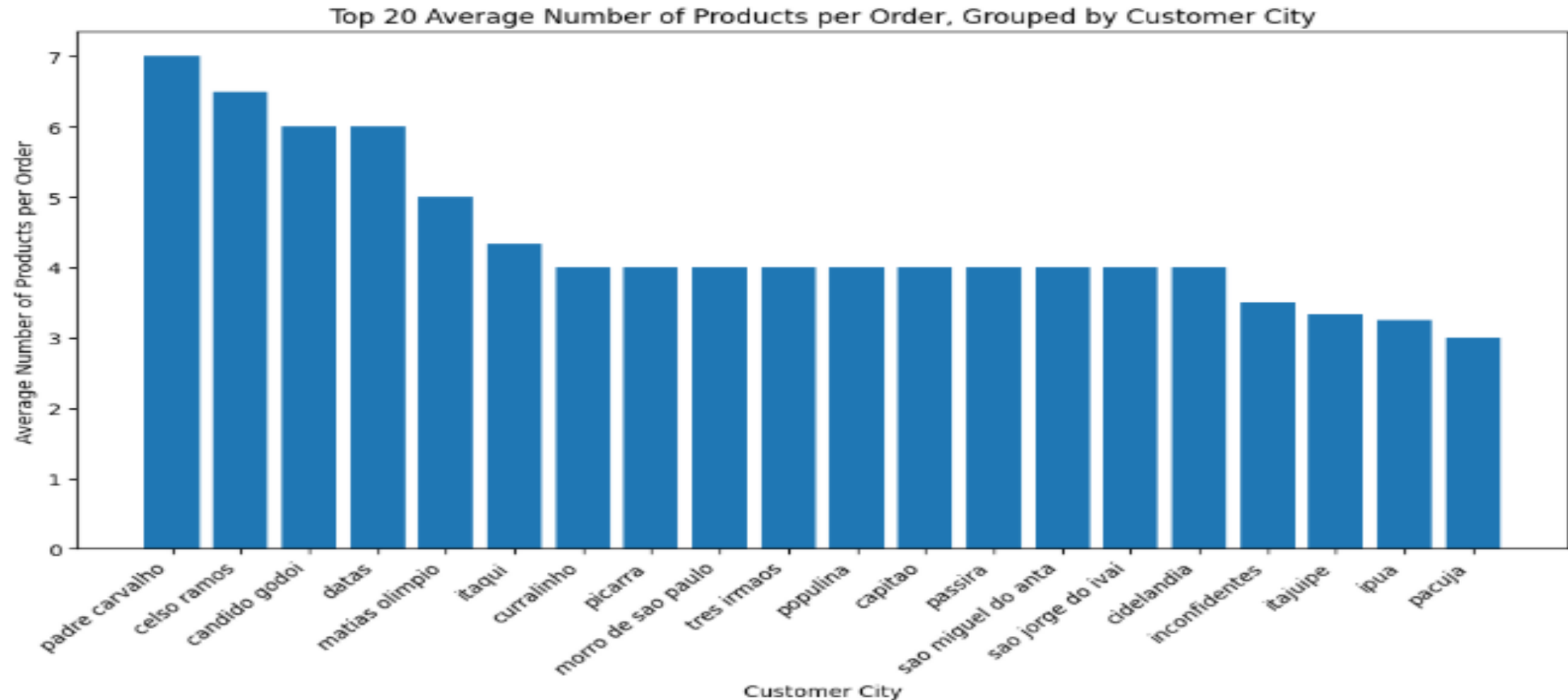
avg_products_per_city = avg_products_per_city.groupby('customer_city')['avg_products_per_order_per_city'].mean().reset_index()
avg_products_per_city = avg_products_per_city.sort_values(by='avg_products_per_order_per_city', ascending=False)
avg_products_per_city

print("Average number of products per order, grouped by customer city:")
avg_products_per_city
```

Average number of products per order, grouped by customer city:

	customer_city	avg_products_per_order_per_city
2625	padre carvalho	7.0
909	celso ramos	6.5
758	candido godoi	6.0
1156	datas	6.0
2269	matias olimpio	5.0
...	...	...
1949	jataizinho	0.0
1227	doutor mauricio cardoso	0.0

Chart for average number of products per order, grouped by customer city.



SQL Answer

```
57      /*Find the average number of products per order, grouped by customer city.*/
58 •    with count_per_order as
59      (select o.order_id, o.customer_id, count(oi.order_id) as order_count
60       from orders as o
61       join order_items as oi
62       on o.order_id= oi.order_id
63       group by o.order_id, o.customer_id)
64
65      select c.customer_city, round(avg(cp.order_count), 2) as avg_order
66      from customers as c
```

< | **Result Grid** | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: |

	customer_city	avg_order
▶	Padre Carvalho	7.00
	Celso Ramos	6.50
	Candido Godoi	6.00
	Datas	6.00
	Matias Olimpio	5.00
	Morro De Sao Paulo	4.00
	Curralinho	4.00
	Turkey	4.00

Result 15 x

C. Calculate the percentage of total revenue contributed by each product category

C. Calculate the percentage of total revenue contributed by each product category

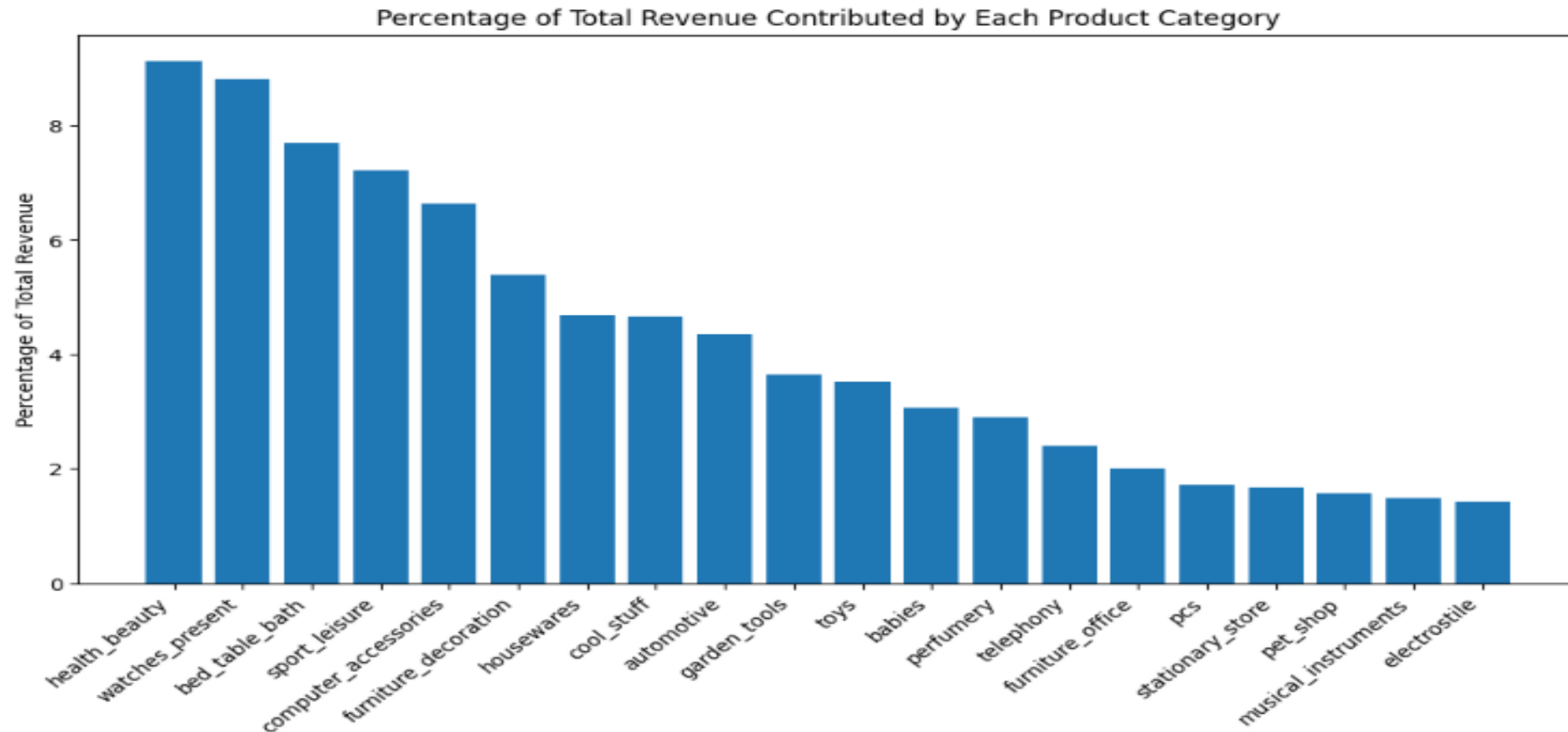
```
rev_product_category= df_merged.groupby('product_category')['price'].sum().reset_index(name= 'revenue')
rev_product_category['product_category']= rev_product_category['product_category'].astype(str).str.lower().str.replace(' ', '_').str.strip()
rev_product_category

total_revenue= df_merged['price'].sum()

rev_product_category['percentage_revenue']= (rev_product_category['revenue']/total_revenue)*100
rev_product_category= rev_product_category.sort_values(by='percentage_revenue', ascending=False).reset_index(drop=True)
rev_product_category
```

	product_category	revenue	percentage_revenue
0	health_beauty	1297490.77	9.131311
1	watches_present	1253143.30	8.819208
2	bed_table_bath	1092551.02	7.689012
3	sport_leisure	1023996.34	7.206547
4	computer_accessories	942277.57	6.631438
...	...	...	...
69	flowers	1110.04	0.007812

Chart for percentage of total revenue contributed by each product category



SQL Answer

```
72  /*Calculate the percentage of total revenue contributed by each product category*/
73  • select p.product_category, round(sum(oi.price),2) as revenue,
74      round((sum(oi.price)/ (select sum(price) from order_items))*100, 2) as revenue_per_category
75  from products as p
76  join order_items as oi
77  on p.product_id= oi.product_id
78  group by p.product_category
79  order by revenue_per_category desc;
```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	product_category	revenue	revenue_per_category
▶	Health Beauty	1258681.34	9.26
	Watches Present	1205005.68	8.87
	Bed Table Bath	1036988.68	7.63
	Sport Leisure	988048.97	7.27
	Computer Accessories	911954.32	6.71
	Furniture Decoration	729762.49	5.37
	Cool Stuff	635290.85	4.67
	Housewares	632248.66	4.65
	Automotive	592720.11	4.36
	Garden Tools	485256.46	3.57

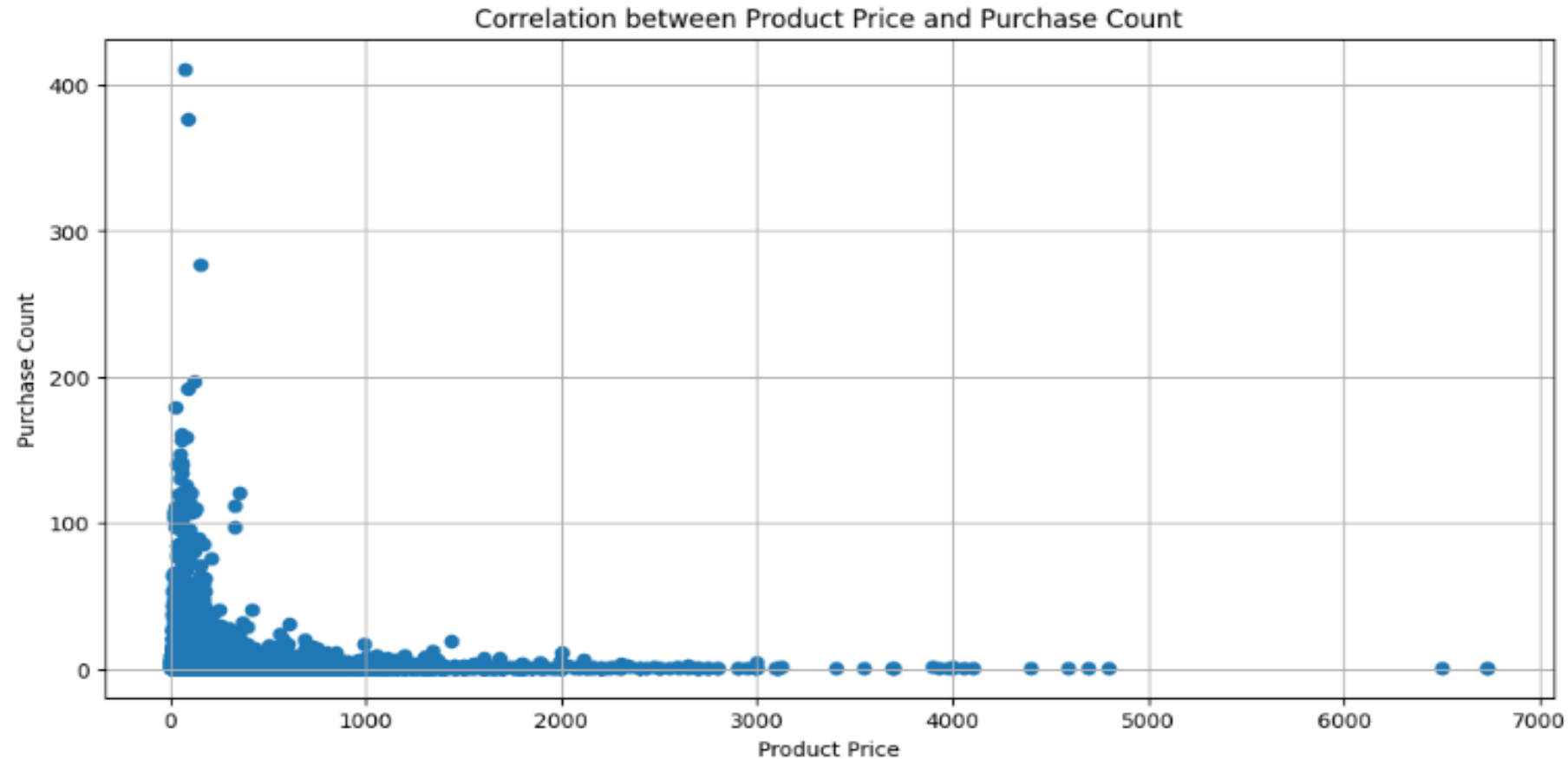
D. Identify the correlation between product price and the number of times a product has been purchased

D. Identify the correlation between product price and the number of times a product has been purchased

```
product_sales = (  
    df_merged  
    .groupby(['product_id', 'price'])  
    .agg(purchase_count=('order_id', 'count'))  
    .reset_index()  
  
    correlation = product_sales['price'].corr(product_sales['purchase_count'])  
    print(f"Pearson correlation between product price and purchase count: {correlation}")
```

Pearson correlation between product price and purchase count: -0.044187682605407216

Chart for correlation between product price and the number of times a product has been purchased



SQL Answer

```
81      /*Identify the correlation between product price and the number of times a product has been purchased*/
82
83      • select p.product_category, count(oi.product_id) as count_of_products,
84         round(avg(oi.price), 2) as avg_price
85      from products as p
86      join order_items as oi
87      on p.product_id= oi.product_id
88      group by p.product_category;
```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	product_category	count_of_products	avg_price
▶	Health Beauty	9670	130.16
	Sport Leisure	8641	114.34
	Cool Stuff	3796	167.36
	Computer Accessories	7827	116.51
	Watches Present	5991	201.14
	Housewares	6964	90.79
	Electronics	2767	57.91
	Toys	4117	117.55
	Bed Table Bath	11115	93.3
	Games Consoles	1137	138.49

E. Calculate the total revenue generated by each seller and rank them by revenue.

E. Calculate the total revenue generated by each seller, and rank them by revenue.

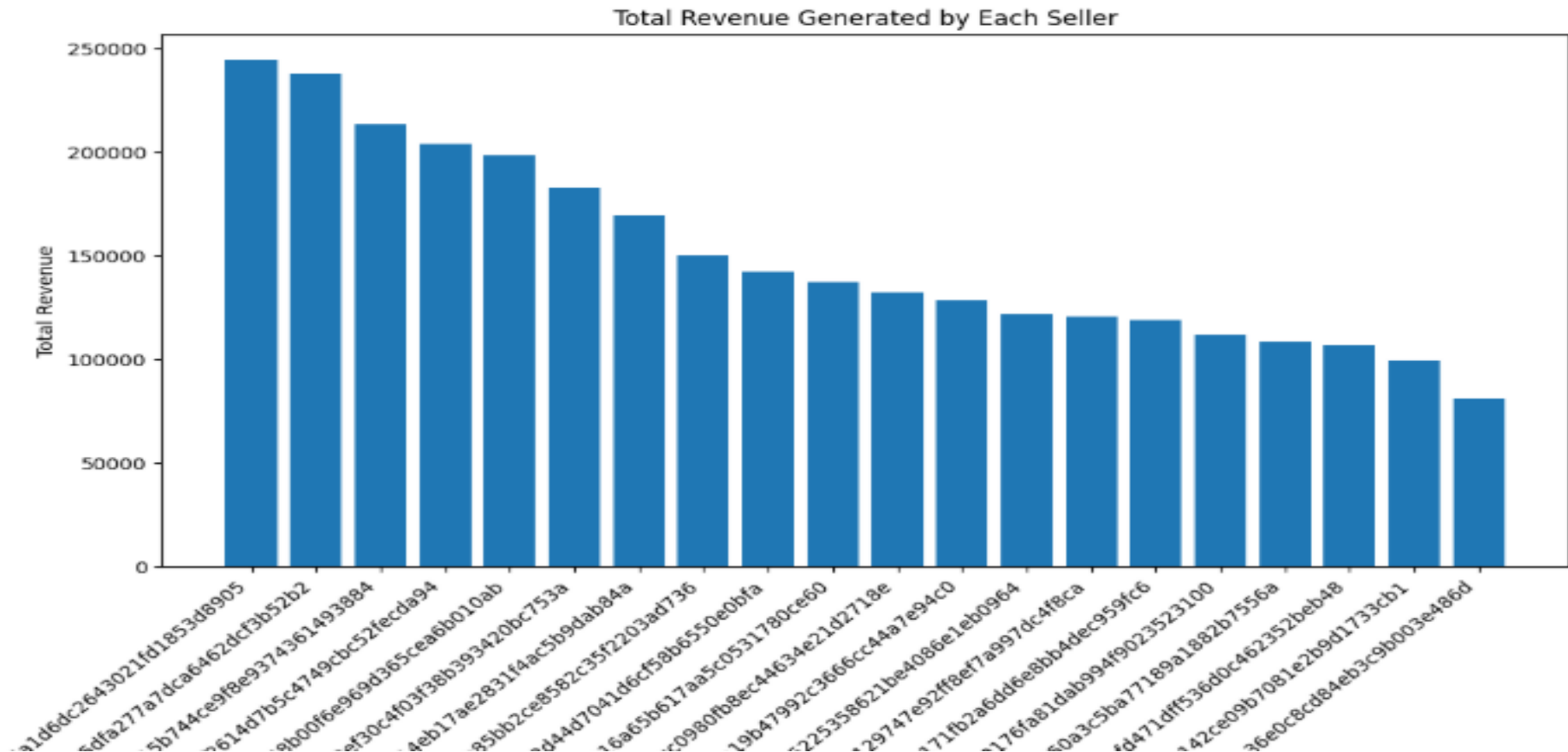
```
▶ seller_revenue = df_merged.groupby('seller_id')['price'].sum().reset_index(name='total_revenue')

seller_revenue['rank'] = seller_revenue['total_revenue'].rank(ascending=False).astype(int)
seller_revenue = seller_revenue.sort_values(by='total_revenue', ascending=False)
seller_revenue
```

```
***
```

	seller_id	total_revenue	rank
1013	53243585a1d6dc2643021fd1853d8905	244627.55	1
857	4869f7a5dfa277a7dca6462dcf3b52b2	237867.23	2
881	4a3ca9315b744ce9f8e9374361493884	213299.57	3
3024	fa1c13f2614d7b5c4749cbc52fecda94	203984.22	4
1535	7c67e1448b00f6e969d365cea6b010ab	198194.33	5
...	...	...	...
2087	ad14615bdd492b01b0d97922e87cb87f	8.25	3091
1370	702835e4b785b67a084280efca355756	7.60	3092
373	1fa2d3def6adfa70e58c276bb64fe5bb	6.90	3093
4165	77128dec4bec4878c37ab7d6160d6f26	6.50	3094

Chart for the total revenue generated by each seller and rank them by revenue



SQL Answer

```
90      /*Calculate the total revenue generated by each seller, and rank them by revenue*/
91      • select seller_id, total_sales, rank() over (order by total_sales) as ranks
92      from (
93      select s.seller_id, sum(oi.price) as total_sales
94      from sellers as s
95      join order_items as oi
96      on s.seller_id = oi.seller_id
97      group by s.seller_id) t
98      order by ranks;
```

Result Grid			
Filter Rows:			
Export: 			
Wrap Cell Content: 			
Fetch rows: 			
	seller_id	total_sales	ranks
▶	cf6f6bc4df3999b9c6440f124fb2f687	3.5	1
	77128dec4bec4878c37ab7d6169d6f26	6.5	2
	1fa2d3def6adfa70e58c276bb64fe5bb	6.9	3
	702835e4b785b67a084280efca355756	7.6	4
	34aefe746cd81b7f3b23253ea28bef39	8	5
	ad14615bdd492b01b0d97922e87cb87f	8.25	6
	4965a7002cca77301c82d3f91b82e1a9	8.49	7
	0f94588695d71662beec8d883ffacf09	9	8
	c18309219e789960add0b2255ca4b091	9.9	9
	95cca791657aabeff15a07eb152d7841	9.99	10

Result 18



ADVANCED PROBLEMS

A. Calculate the moving average of order values for each customer over their order history.

- ▼ A. Calculate the moving average of order values for each customer over their order history.

```
[ ] ▶ df_merged_sorted = df_merged.sort_values(by=['customer_unique_id', 'order_purchase_timestamp'])

df_merged_sorted['moving_avg_order_value'] = df_merged_sorted.groupby('customer_unique_id')['payment_value'].transform(lambda x: x.rolling(window=3, min_periods=1).mean())

print("DataFrame with moving average of order values:")
display(df_merged_sorted[['customer_unique_id', 'order_purchase_timestamp', 'payment_value', 'moving_avg_order_value']].head(10))
```

- ▼ ... DataFrame with moving average of order values:

	customer_unique_id	order_purchase_timestamp	payment_value	moving_avg_order_value
62840	0000366f3b9a7992bf8c76cfd3221e2	2018-05-10 10:56:27	141.90	141.90
87880	0000b849f77a49e4a4ce2b2a4ca5be3f	2018-05-07 11:11:27	27.19	27.19
31523	0000f48a3911fa3c0805444483337064	2017-03-10 21:05:03	86.22	86.22
117277	0000f6ccb0745a6a4b88665a16c9f078	2017-10-12 20:29:41	43.62	43.62
49537	0004aac84e0df4da2b147fca70cf8255	2017-11-14 19:45:42	196.89	196.89
93361	0004bd2a26a76fe21f786e4fbd80607f	2018-04-05 19:33:16	166.98	166.98
88848	00050ab1314c0e55a6ca13cf7181fecf	2018-04-20 12:57:23	35.38	35.38
8092	00053a61a98854899e70ed204dd4baf6	2018-02-28 11:15:41	419.18	419.18
8093	00053a61a98854899e70ed204dd4baf6	2018-02-28 11:15:41	419.18	419.18
84664	0005e1862207bf8ccc02e4228effd9a0	2017-03-04 23:32:12	150.12	150.12

B. Calculate the cumulative sales per month for each year

B. Calculate the cumulative sales per month for each year

```
df_merged['order_year'] = df_merged['order_purchase_timestamp'].dt.year
df_merged['order_month'] = df_merged['order_purchase_timestamp'].dt.month

monthly_sales = df_merged.groupby(['order_year', 'order_month'])['price'].sum().reset_index()

cumulative_sales_per_year_month = monthly_sales.groupby('order_year')['price'].cumsum().reset_index(name='cumulative_sales')
cumulative_sales_per_year_month = pd.merge(monthly_sales[['order_year', 'order_month']], cumulative_sales_per_year_month, left_index=True, right_index=True)

cumulative_sales_per_year_month
```

```
***
   order_year  order_month  index  cumulative_sales
0      2016           9         0         287.36
1      2016          10         1        51209.59
2      2016          12         2        51220.49
3      2017           1         3       128637.99
4      2017           2         4      389391.85
5      2017           3         5       785842.89
6      2017           4         6      1177627.31
7      2017           5         7      1722987.96
8      2017           6         8      2175999.37
9      2017           7         9      2709616.21
10     2017           8        10     3311025.88
11     2017           9        11     3973302.27
12     2017          10        12     4667109.87
13     2017          11        13     5713237.71
```

SQL Answer

```
100      /*Calculate the cumulative sales per month for each year*/
101
102 •    with monthly_sales as
103      (select extract(year from o.order_purchase_timestamp) as sales_year,
104             extract(month from o.order_purchase_timestamp) as sales_month,
105             round(sum(oi.price), 2) as Monthly_Sales
106             from orders as o
107             join order_items oi on o.order_id = oi.order_id)
```

Result Grid Filter Rows: Export: Wrap Cell Content:				
	sales_year	sales_month	Monthly_Sales	cumulative_sales
▶	2016	9	267.36	267.36
	2016	10	49507.66	49775.02
	2016	12	10.9	49785.92
	2017	1	120312.87	120312.87
	2017	2	247303.02	367615.89
	2017	3	374344.3	741960.19
	2017	4	359927.23	1101887.42
	2017	5	506071.14	1607958.56
	2017	6	433038.6	2040997.16
	2017	7	498031.48	2539028.64

C. Calculate the year-over-year growth rate of total sales

- ✓ C. Calculate the year-over-year growth rate of total sales

```
1 yearly_sales = df_merged.groupby('order_year')['price'].sum().reset_index()
   yearly_sales.rename(columns={'price': 'total_sales'}, inplace=True)

   # Calculate Year-over-Year Growth Rate
   yearly_sales['previous_year_sales'] = yearly_sales['total_sales'].shift(1)
   yearly_sales['yoy_growth_rate'] = ((yearly_sales['total_sales'] - yearly_sales['previous_year_sales']) / yearly_sales['previous_year_sales']) * 100
   yearly_sales['yoy_growth_rate'] = yearly_sales['yoy_growth_rate'].fillna(0)

   print("Year-over-Year Growth Rate of Total Sales:")
   display(yearly_sales)
```

- ✓ Year-over-Year Growth Rate of Total Sales:

	order_year	total_sales	previous_year_sales	yoy_growth_rate
0	2016	51220.49	NaN	0.000000
1	2017	6483893.86	51220.49	12558.789207
2	2018	7674135.96	6483893.86	18.356903

5. If the year-over-year growth rate is negative, then the sales are decreasing.

SQL Answer

```
118      /*Calculate the year-over-year growth rate of total sales*/
119
120 •   with year_sales as
121     (select extract(year from o.order_purchase_timestamp) as sales_year, round(sum(oi.price), 2) as total_sales
122      from orders as o
123       join order_items as oi
124       on o.order_id = oi.order_id
125      group by extract(year from o.order_purchase_timestamp))
```

Result Grid				
Filter Rows:		Export:		Wrap Cell Content:
	curr_year_sales	total_sales	previous_year_sales	YoY_Sale_percentage
▶	2016	49785.92	NULL	NULL
	2017	6155806.98	49785.92	12264.55
	2018	7386050.8	6155806.98	19.99

D. Identify the top 3 customers who spent the most money in each year

D. Identify the top 3 customers who spent the most money in each year.

```
customer_yearly_spending = df_merged.groupby(['order_year', 'customer_unique_id'])['price'].sum().reset_index(name='total_spent').sort_values(by='total_spent', ascending=False)
customer_yearly_spending

top3_each_year = customer_yearly_spending.groupby('order_year').head(3).reset_index(drop=True).sort_values(by=['order_year', 'total_spent'], ascending=[True, False])
top3_each_year
```

```
***
```

	order_year	customer_unique_id	total_spent
6	2016	fdaa290acb9eeacb66fa7f979baa6803	1399.00
7	2016	753bc5d6efa9e49a03e34cf521a9e124	1299.99
8	2016	b92a2e5e8a6eabcc80882c7d68b2c70b	1199.00
0	2017	0a0a92112bd4c708ca5fde585afaa872	13440.00
1	2017	8af7ac63b2efbcbdd88e5b11505e8098a	11383.95
2	2017	c4b224d2c784bae11ae98b6ae9f2454c	10856.10
3	2018	85963fd37bfd387aa6d915d8a1065486	9888.00
4	2018	be74c431147c32ab2d7c7cef5e4a995f	9520.14
5	2018	58483a1c055dfb600f57c5b867174542	7798.00

SQL Answer

```
137      /*Identify the top 3 customers who spent the most money in each year*/
138  • WITH customer_yearly_spend AS (
139      SELECT
140          YEAR(o.order_purchase_timestamp) AS sales_year,
141          o.customer_id,
142          ROUND(SUM(oi.price), 2) AS total_spent
143      FROM orders o
```

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
	sales_year	customer_id	total_spent
▶	2016	a9dc96b027d1252bbac0a9b72d837fc6	1399
	2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99
	2016	4a06381959b6670756de02e07b83815f	1199
	2017	1617b1357756262bfa56ab541c47bc16	13440
	2017	c6e2731c5b391845f6800c97401a43a9	6735
	2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499
	2018	ec5b2ba62e574342386871631fafd3fc	7160
	2018	f48d464a0baaea338cb25f816991ab1f	6729
	2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.9



Thank You
